Capítulo 1

Projeto de Programas e Estruturas de Ramificação

OBJETIVOS

- Aprender os conceitos de projeto de cima para baixo (top-down) e decomposição.
- Aprender sobre pseudocódigo e fluxogramas, e por que eles devem ser usados.
- Saber como criar e usar constantes e variáveis `LÓGICAS`.
- Aprender sobre operadores lógicos relacionais e combinacionais, e como eles se encaixam na hierarquia de operações.
- Saber como usar a construção `SE` (IF).
- Saber como usar a construção `ESCOLHA CASO` (SELECT CASE).

Nos capítulos anteriores, desenvolvemos vários programas Fortran completos e funcionais. No entanto, todos os programas eram muito simples, consistindo em uma série de declarações Fortran que eram executadas uma após a outra em uma ordem fixa. Esses programas são chamados de programas sequenciais. Eles leem dados de entrada, processam-nos para produzir uma resposta desejada, imprimem a resposta e terminam. Não há como repetir seções do programa mais de uma vez e não há como executar seletivamente apenas certas partes do programa, dependendo dos valores dos dados de entrada.

Nos próximos dois capítulos, introduziremos uma série de declarações Fortran que nos permitem controlar a ordem em que as declarações são executadas em um programa. Existem duas categorias amplas de declarações de controle: **desvios** (branches), que selecionam seções específicas do código para executar, e **laços** (loops), que fazem com que seções específicas do código sejam repetidas. Os desvios serão introduzidos neste capítulo e os laços serão abordados no Capítulo 4.

Com a introdução de desvios e laços, nossos programas se tornarão mais complexos e será mais fácil cometer erros. Para ajudar a evitar erros de programação, introduziremos um procedimento formal de projeto de programas baseado na técnica conhecida como projeto de cima para baixo (top-down). Também introduziremos duas ferramentas comuns de desenvolvimento de algoritmos, fluxogramas e pseudocódigo.

Após introduzir o processo de projeto de programas, introduziremos o tipo de dados lógico e as operações que os produzem. Expressões lógicas são usadas para controlar muitas declarações de desvio, então aprenderemos sobre elas antes de estudar os desvios.

Finalmente, estudaremos os vários tipos de declarações de desvio Fortran.

1.1 Introdução às Técnicas de Projeto de Cima para Baixo

Suponha que você seja um engenheiro trabalhando na indústria e que precise escrever um programa Fortran para resolver algum problema. Como você começa?

Ao receber um novo problema, há uma tendência natural de sentar-se em um terminal e começar a programar sem "perder" muito tempo pensando sobre isso primeiro. Muitas vezes, é possível se safar com essa abordagem de programação "improvisada" para problemas muito pequenos, como muitos dos exemplos deste livro. No mundo real, no entanto, os problemas são maiores e um programador que tentar essa abordagem ficará irremediavelmente atolado. Para problemas maiores, vale a pena pensar completamente sobre o problema e a abordagem que você vai adotar antes de escrever uma única linha de código.

Nesta seção, apresentaremos um processo formal de projeto de programas e, em seguida, aplicaremos esse processo a todas as principais aplicações desenvolvidas no restante do livro. Para alguns dos exemplos simples que faremos, o processo de projeto parecerá exagerado. No entanto, à medida que os problemas que resolvemos se tornam cada vez maiores, o processo se torna cada vez mais essencial para uma programação bem-sucedida.

Quando eu era estudante de graduação, um dos meus professores gostava de dizer: "Programar é fácil. O difícil é saber o que programar". Seu ponto foi fortemente reforçado para mim depois que saí da universidade e comecei a trabalhar na indústria em projetos de software de maior escala. Descobri que a parte mais difícil do meu trabalho era entender o problema que eu estava tentando resolver. Uma vez que eu realmente entendia o problema, tornava-se fácil dividi-lo em pedaços menores, mais facilmente gerenciáveis, com funções bem definidas e, em seguida, abordar esses pedaços um de cada vez.

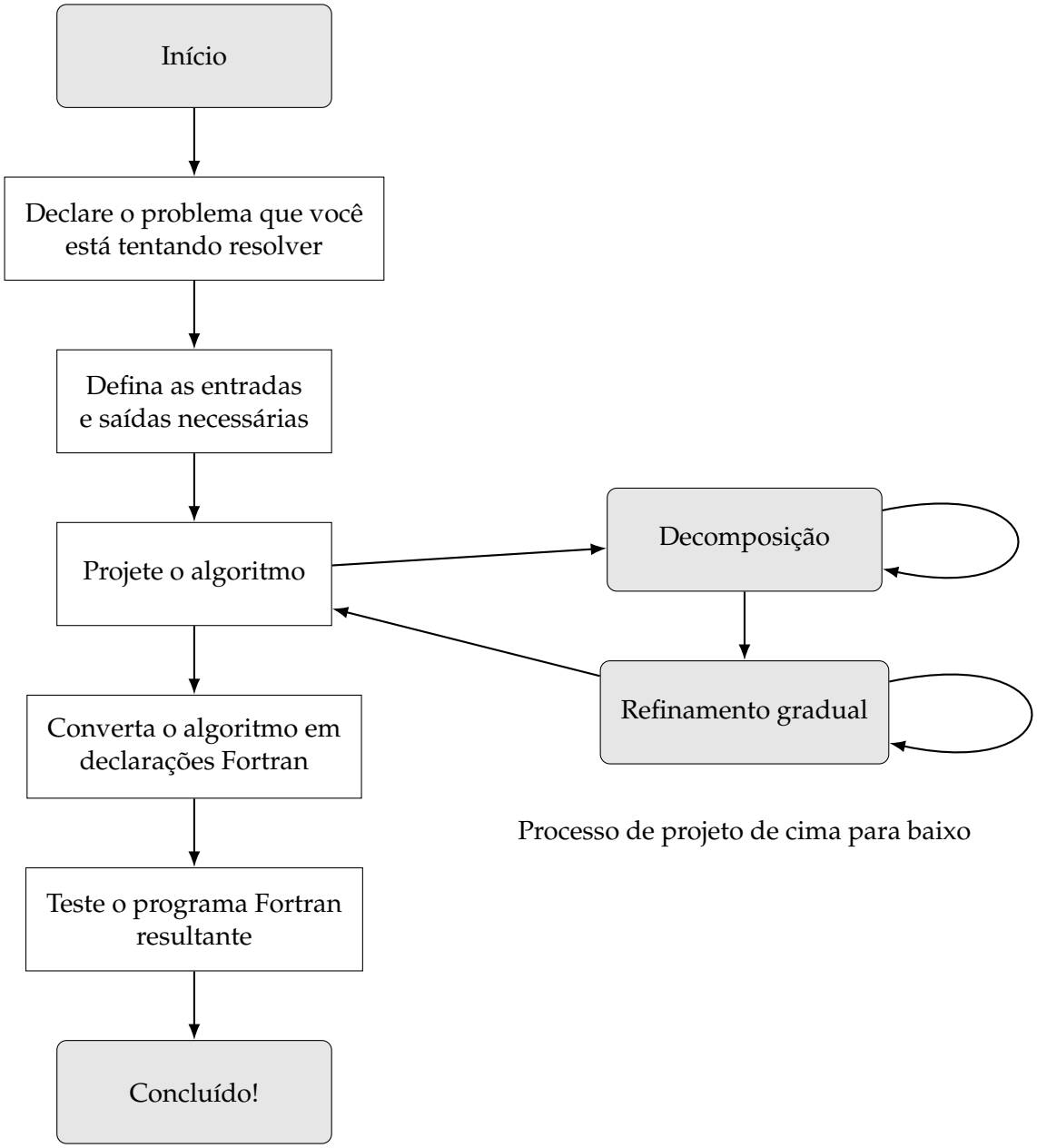
O projeto de cima para baixo é o processo de começar com uma tarefa grande e dividi-la em pedaços menores, mais facilmente compreensíveis (subtarefas) que executam uma parte da tarefa desejada. Cada subtarefa pode, por sua vez, ser subdividida em subtarefas menores, se necessário. Depois que o programa é dividido em pequenos pedaços, cada pedaço pode ser codificado e testado independentemente. Não tentamos combinar as subtarefas em uma tarefa completa até que cada uma das subtarefas tenha sido verificada para funcionar corretamente por si só.

O conceito de projeto de cima para baixo é a base do nosso processo formal de projeto de programas. Agora, vamos apresentar os detalhes do processo, que é ilustrado na Figura 1.1. As etapas envolvidas são:

1. *Declare claramente o problema que você está tentando resolver.*

Os programas geralmente são escritos para suprir alguma necessidade percebida, mas essa necessidade pode não estar articulada claramente pela pessoa que solicita o programa. Por exemplo, um usuário pode solicitar um programa para resolver um sistema de equações lineares simultâneas. Essa solicitação não é clara o suficiente para permitir que um programador projete um programa para atender à necessidade; ele ou ela deve primeiro saber muito mais sobre o problema a ser resolvido. O sistema de equações a ser resolvido é real ou complexo? Qual é o número máximo de equações e incógnitas que o programa deve lidar? Existem simetrias nas equações que podem ser exploradas para facilitar a tarefa? O projetista do programa terá que conversar com o usuário que está solicitando o programa e os dois terão que chegar a uma declaração clara do que exatamente estão tentando realizar. Uma declaração clara

Figura 1.1: O processo de projeto de programas usado neste livro.



do problema evitará mal-entendidos e também ajudará o projetista do programa a organizar adequadamente seus pensamentos. No exemplo que estávamos descrevendo, uma declaração adequada do problema poderia ter sido:

> Projetar e escrever um programa para resolver um sistema de equações lineares simultâneas com coeficientes reais e com até 20 equações em 20 incógnitas.

2. *Defina as entradas necessárias para o programa e as saídas a serem produzidas pelo programa.*

As entradas para o programa e as saídas produzidas pelo programa devem ser especificadas para que o novo programa se encaixe adequadamente no esquema geral de processamento. No exemplo acima, os coeficientes das equações a serem resolvidas provavelmente estão em alguma ordem preexistente e nosso novo programa precisa ser capaz de lê-los nessa ordem. Da mesma forma, ele precisa produzir as respostas exigidas pelos programas que podem segui-lo no esquema geral de processamento e gravar essas respostas no formato necessário para os programas seguintes.

3. *Projete o algoritmo que você pretende implementar no programa.*

Um **algoritmo** é um procedimento passo a passo para encontrar a solução para um problema. É nesta etapa do processo que as técnicas de projeto de cima para baixo entram em jogo. O projetista procura divisões lógicas dentro do problema e o divide em subtarefas ao longo dessas linhas. Esse processo é chamado de *decomposição*. Se as subtarefas forem grandes, o projetista pode dividi-las em sub-subtarefas ainda menores. Esse processo continua até que o problema seja dividido em muitas partes pequenas, cada uma das quais executa um trabalho simples e claramente compreensível.

Depois que o problema é decomposto em pequenos pedaços, cada pedaço é ainda mais refinado por meio de um processo chamado *refinamento gradual*. No refinamento gradual, um projetista começa com uma descrição geral do que o pedaço de código deve fazer e, em seguida, define as funções do pedaço com detalhes cada vez maiores até que sejam específicos o suficiente para serem transformados em declarações Fortran. O refinamento gradual é geralmente feito com pseudocódigo, que será descrito na próxima seção.

Muitas vezes, é útil resolver um exemplo simples do problema manualmente durante o processo de desenvolvimento do algoritmo. Se o projetista entender os passos que ele ou ela seguiu para resolver o problema manualmente, ele ou ela estará mais apto a aplicar a decomposição e o refinamento gradual ao problema.

4. *Transforme o algoritmo em declarações Fortran.*

Se o processo de decomposição e refinamento for realizado corretamente, esta etapa será muito simples. Tudo o que o programador terá que fazer é substituir o pseudocódigo pelas declarações Fortran correspondentes, uma a uma.

5. *Teste o programa Fortran resultante.*

Esta etapa é a verdadeira chave. Os componentes do programa devem primeiro ser testados individualmente, se possível, e então o programa como um todo deve ser testado. Ao testar um programa, devemos verificar se ele funciona corretamente para todos os conjuntos de dados de entrada legais. É muito comum que um programa seja escrito, testado com algum conjunto de dados padrão e liberado para uso, apenas para descobrir que ele produz respostas erradas (ou falha) com um conjunto de dados de entrada diferente. Se o algoritmo implementado em um programa incluir diferentes desvios, devemos testar todos os possíveis desvios para confirmar se o programa opera corretamente sob todas as circunstâncias possíveis.

Programas grandes normalmente passam por uma série de testes antes de serem liberados para uso geral (veja a Figura 3-2). O primeiro estágio de teste às vezes é chamado de teste de unidade. Durante o teste de unidade, as subtarefas individuais do programa são testadas separadamente para confirmar se elas funcionam corretamente. O programador geralmente escreve pequenos programas chamados "stubs"ou "drivers de teste"para executar o código em teste e ver se o código está retornando os resultados adequados. Isso verifica a operação das subtarefas em um nível básico antes que elas sejam combinadas em grupos maiores.